❸ Mixing C# and HTML allows you to create HTML dynamically at runtime.

The pure HTML sections in this template are in the angle brackets. The Razor engine copies this HTML directly to the output, unchanged, as though you were writing a normal HTML file.

NOTE The ability of Razor syntax to know when you are switching between HTML and C# can be both uncanny and infuriating at times. I discuss how to control this transition in section 17.3.

As well as HTML, you can see several C# statements in there. The advantage of being able to, for example, use a `for` loop rather than having to explicitly write out each `<li>` element should be self-evident. I'll dive a little deeper into more of the C# features of Razor in the next section. When rendered, the template in listing 17.2 produces the following HTML.

Listing 17.3 HTML output produced by rendering a Razor template

```
<h1>Tasks to complete</h1>     ❶
<ul>                           ❶
  <li>0 - Buy milk</li>        ❷
  <li>1 - Buy eggs</li>        ❷
  <li>2 - Buy bread</li>       ❷
</ul>                          ❸
```

❶ HTML from the Razor template is written directly to the output.

❷ The <li> elements are generated dynamically by the for loop, based on the data provided.

❸ HTML from the Razor template is written directly to the output.

As you can see, the final output of a Razor template after it's rendered is simple HTML. There's nothing complicated left, only straight HTML markup that can be sent to the browser and rendered. Figure 17.3 shows how a browser would render it.